

SiyaBusa ERP

Technical Implementation

Investor Documentation

Masaphokati Technologies (Pty) Ltd

Document Version: 1.0

Effective Date: January 29, 2026

Department: Executive

Classification: Confidential

WE RULE. WE GOVERN. WE EMPOWER.

Table of Contents

1. AWS DEPLOYMENT STATUS

2. FRONTEND BACKEND INTEGRATION

3. DATABASE MIGRATION SUCCESS

AWS DEPLOYMENT STATUS

DEPLOYMENT STATUS - AetherOS ERP on AWS

✓ COMPLETED (Steps 1-8)

1. EC2 Instance ✓

- ****Public IP****: `51.21.219.35`
- ****Instance ID****: `i-0b20fd06fae7e84b1`
- ****Instance Type****: t3.micro (FREE tier)
- ****Region****: eu-north-1
- ****Status****: ✓ Running

2. RDS PostgreSQL Database ✓

- ****Endpoint****: `aetheros-erp-db.cxoqqoowwgxt.eu-north-1.rds.amazonaws.com`
- ****Port****: `5432`
- ****Database Name****: `aetheros\_erp`
- ****Instance Type****: db.t3.micro (FREE tier)
- ****Status****: ✓ Running
- ****Tables Created****: users, chart_of_accounts, journal_entries, journal_entry_lines

3. Backend API ✓

- ****URL****: `http://51.21.219.35:3000`
- ****Status****: ✓ LIVE and accessible
- ****Health Check****: `curl http://51.21.219.35:3000/health` →
`{"status":"OK","message":"Server is running"}`
- ****Process Manager****: PM2 (auto-restart enabled)
- ****Environment****: Production
- ****Database****: Connected to RDS ✓

4. Frontend Build

- **Status**:  Built successfully
 - **Size**: 1.4 MB
 - **Location**: `/frontend/dist/`
 - **API URL**: `http://51.21.219.35:3000`
-

REMAINING STEPS (Steps 9-10)

Step 9: Deploy Frontend to S3 (10 minutes)

Since AWS CLI is not installed, use **AWS Console** (web browser):

1. **Go to S3** in AWS Console
2. **Create Bucket**: - Click **Create bucket** - Bucket name: `aetheros-erp-frontend-YOUR\_AWS\_ACCOUNT\_ID` (must be globally unique) - Region: **eu-north-1** (same as your EC2) - **Uncheck** "Block all public access"  - Check "I acknowledge that the current settings might result in this bucket and the objects within becoming public" - Click **Create bucket**
3. **Upload Frontend Files**: - Open your new bucket - Click **Upload** - Click **Add files** - Select ALL files from: `/Users/sibusisomavuso/Desktop/Worldclass ERP Software /frontend/dist/` - **Important**: Upload the contents of `dist/`, not the `dist/` folder itself - Files to upload: * `index.html` * `vite.svg` * The entire `assets/` folder - Click **Upload** - Wait for completion
4. **Enable Static Website Hosting**: - Go to **Properties** tab - Scroll to **Static website hosting** - Click **Edit** - Select **Enable** - Index document: `index.html` - Error document: `index.html` - Click **Save changes** - **Copy the Bucket website endpoint** (example: `http://aetheros-erp-frontend-123456.s3-website-eu-north-1.amazonaws.com`)
5. **Set Bucket Policy for Public Access**: - Go to **Permissions** tab - Scroll to **Bucket policy** - Click **Edit** - Paste this (replace `YOUR-BUCKET-NAME`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

- Click **Save changes**

6. **Done!** Your frontend is now live at the S3 website endpoint.

Step 10: Test Your Application (5 minutes)

1. **Open the S3 website URL** in your browser - Example: `http://aetheros-erp-frontend-123456.s3-website-eu-north-1.amazonaws.com`
2. **You should see:** AetherOS ERP login page with: - Oracle blue header - Microsoft purple CoPilot button - Professional login form
3. **Test Navigation:** - Click around the menu - Go to **Financial Management** - Check the enhanced pages with purple gradients: * Dashboard (KPI cards) * Journal Entries (with secondary sidebar) * Trial Balance (with filters) * Chart of Accounts (with secondary sidebar) * Financial Statements Hub
4. **Verify API Connection:** - Open browser console (F12) - Navigate through pages - Check Network tab - you should see API calls to `http://51.21.219.35:3000`

Your Live URLs

- **Backend API:** `http://51.21.219.35:3000`
- **Frontend:** `http://aetheros-erp-frontend-[YOUR-ID].s3-website-eu-north-1.amazonaws.com`
- **Database:** `aetheros-erp-db.cxoqqpwwgxt.eu-north-1.rds.amazonaws.com:5432`

Monthly Cost

Current Setup (First 12 Months): \$0/month

- EC2 t3.micro: FREE (750 hours/month)
- RDS db.t3.micro: FREE (750 hours/month + 20GB)
- S3 Storage: FREE (5GB + 20,000 requests)
- Data Transfer: FREE (15GB outbound)

After 12 Months: ~\$28-35/month

How to Update Your Code

Update Backend:

```
# On your Mac
cd backend
npm run build
scp -i ~/.ssh/aetheros-aws.pem -r dist/ ec2-user@51.21.219.35:~/backend/

# SSH into EC2
ssh -i ~/.ssh/aetheros-aws.pem ec2-user@51.21.219.35

# Restart backend
pm2 restart aetheros-backend
pm2 logs aetheros-backend --lines 20
```

Update Frontend:

```
# On your Mac
cd frontend
npm run build

# Upload dist/ contents to S3 via AWS Console
# Or install AWS CLI and use:
# aws s3 sync dist/ s3://your-bucket-name --delete
```

Backend not responding:

```
ssh -i ~/.ssh/aetheros-aws.pem ec2-user@51.21.219.35
pm2 status
pm2 logs aetheros-backend
pm2 restart aetheros-backend
```

Database connection issues:

```
ssh -i ~/.ssh/aetheros-aws.pem ec2-user@51.21.219.35
psql "postgresql://postgres:caxMex-0putca-dyjnah@aetheros-erp-db.cxoqqoowgxt.eu-north-1.
```

Frontend blank page:

- Check browser console for errors
- Verify S3 bucket policy is set correctly
- Ensure static website hosting is enabled
- Check that `index.html` is in the root of the bucket

Security Notes

 **Important:** Your backend API is currently accessible via HTTP (not HTTPS) on port 3000. For production use, you should:

1. Set up an Application Load Balancer with SSL certificate
2. Or use CloudFront with your backend behind an ALB
3. Or restrict backend port 3000 to only CloudFront IP ranges

 **Database Password:** Currently stored in plain text in `.env`. For production:

- Use AWS Secrets Manager
 - Or AWS Systems Manager Parameter Store
-

Next Steps (Optional)

1. ****Add HTTPS**** with CloudFront distribution
 2. ****Custom Domain**** with Route 53
 3. ****Email Service**** with AWS SES
 4. ****File Storage**** with S3 for uploads
 5. ****Monitoring**** with CloudWatch dashboards
 6. ****Backups**** - RDS automated backups already enabled (7 days)
 7. ****SSL Certificate**** - Free with AWS Certificate Manager
-

Congratulations!

You've successfully deployed AetherOS ERP to AWS using the FREE tier!

Your enterprise-grade ERP system is now:

-  Running on production infrastructure
-  Using managed PostgreSQL database
-  Scalable and reliable
-  Costing \$0/month for 12 months

System is 90% complete! Just upload frontend to S3 and you're done! 

Last updated: November 8, 2025 - 02:50 AM SAST

FRONTEND BACKEND INTEGRATION

Frontend-Backend Integration Guide

Current Status: MOCK DATA → Moving to REAL API CALLS

Problem Statement

Your ERP system has:

-  Beautiful frontend with all modules working
-  Backend API with 13 workspace controllers deployed
-  ****Frontend NOT connected to backend (using mock data)****
-  ****No centralized API service****
-  ****No authentication headers on API calls****

Solution Implemented

1. Centralized API Service

File Created: `/frontend/src/services/api.service.ts`

Features:

-  Single source of truth for API_BASE_URL
-  Automatic authentication headers (Bearer token)
-  Automatic workspace/tenant context (X-Workspace-ID, X-Tenant-ID)
-  Consistent error handling
-  Auto-redirect to login on 401 errors
-  Pre-built functions for all 13 workspaces

Usage Example: ```typescript import { workspaceApi } from '../services/api.service';`

`// Get logistics dashboard data const data = await workspaceApi.logistics.getDashboard();`

```
// Get sales leads with filters const leads = await workspaceApi.sales.getLeads({ status:
'active' });

// Create new trip const trip = await workspaceApi.logistics.createTrip({ vehicle_id: 1, driver_id:
5, route: 'Johannesburg to Durban' }); ``
```

API Endpoints Mapping

Backend API Structure (51.21.219.35:3000)

```
`` /api/workspaces/financial/* → Financial workspace /api/workspaces/sales/* → Sales
workspace /api/workspaces/purchase/* → Purchase workspace /api/workspaces/inventory/*
→ Inventory workspace /api/workspaces/hr/* → HR workspace /api/workspaces/logistics/* →
Logistics workspace /api/workspaces/compliance/* → Compliance workspace
/api/workspaces/sars/* → SARS workspace /api/workspaces/assets/* → Assets workspace
/api/workspaces/manufacturing/* → Manufacturing workspace /api/workspaces/warehouse/*
→ Warehouse workspace /api/workspaces/admin/* → Admin workspace
/api/workspaces/superadmin/* → Super Admin workspace ``
```

Authentication Flow

...

1. User logs in → POST /api/auth/login Response: { token, user, workspace }
 2. Store in localStorage: - token (JWT) - user (user object) - workspaceId (current workspace) - tenantId (tenant ID)
 3. All subsequent API calls include: Headers: Authorization: Bearer X-Workspace-ID: X-Tenant-ID: ````
-

Testing API Connectivity

Step 1: Access API Test Dashboard

Navigate to: <http://localhost:5173/api-test>

This dashboard will:

-  Test backend connectivity
-  Show authentication status
-  Test all 14 workspace endpoints
-  Display response times
-  Show detailed error messages

Step 2: Check Backend Health

```
```bash curl http://51.21.219.35:3000/api/health ```
```

Expected response: ```json { "status": "ok", "timestamp": "2025-11-16T..." } ```

## Step 3: Test Authentication

```
```bash curl -X POST http://51.21.219.35:3000/api/auth/login
-H "Content-Type: application/json"
-d '{ "email": "admin@aetheros.com", "password": "your-password" }' ```
```

Expected response: ```json { "success": true, "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { ... }, "workspace": { ... } } ```



Module Migration Status

Already Updated (Using API Service)

1. **Logistics Dashboard** - Connected to `/api/workspaces/logistics/dashboard``

Pending Migration (Still Using Mock Data)

2. **Sales Module** - Needs update to use ``workspaceApi.sales.*``
3. **Purchase Module** - Needs update to use ``workspaceApi.purchase.*``
4. **Financial Module** - Needs update to use ``workspaceApi.financial.*``
5. **Inventory Module** - Needs update to use ``workspaceApi.inventory.*``
6. **HR Module** - Needs update to use ``workspaceApi.hr.*``
7. **SARS Sentinel** - Needs update to use ``workspaceApi.sars.*``
8. **Assets Module** - Needs update to use ``workspaceApi.assets.*``
9. **Manufacturing Module** - Needs update to use ``workspaceApi.manufacturing.*``
10. **Warehouse Module** - Needs update to use ``workspaceApi.warehouse.*``

How to Connect a Module to Backend

Example: HR Dashboard

Before (Mock Data): ```typescript const fetchDashboardData = async () => { setLoading(true); try { // Mock data setStats({ total_employees: 127, monthly_payroll: 4856300, // ... }); } catch (error) { console.error('Error:', error); } finally { setLoading(false); } }; ```

After (Real API): ```typescript import { workspaceApi } from '../services/api.service';`

`const fetchDashboardData = async () => { setLoading(true); try { const data = await workspaceApi.hr.getDashboard();`

```
if (data && data.success) {
  setStats(data.data.stats);
} else {
  throw new Error('Failed to fetch dashboard data');
}
```

`} catch (error) { console.error('Error fetching HR dashboard:', error); // Fallback to mock data on error setStats({ total_employees: 127, monthly_payroll: 4856300, // ... }); } finally { setLoading(false); } }; ```

Authentication Setup

Login Flow

1. User enters email/password on Login page
2. Frontend calls: ``authApi.login(email, password)``
3. Backend validates credentials
4. Backend returns: ``{ token, user, workspace }``
5. Frontend stores in localStorage
6. All subsequent API calls include token

Auto-Login (Remember Me)

When page loads, frontend checks: ```typescript const token = localStorage.getItem('token'); if (token) { // Call authApi.me() to validate token const user = await authApi.me(); // If valid, user is logged in // If invalid (401), redirect to login } ```

Environment Configuration

Development (.env.development)

```
```bash VITE_API_URL=http://localhost:3000 VITE_ENV=development VITE_API_DEBUG=true ```
```

### Production (.env.production)

```
```bash VITE_API_URL=http://51.21.219.35:3000 VITE_ENV=production VITE_API_DEBUG=true  
```
```

---

## Common Issues & Solutions

---

### Issue 1: "Network Error" on API Calls

**Cause:** Backend not running or CORS issues

**Check:** ```bash

## Test backend health

---

```
curl http://51.21.219.35:3000/api/health
```

## Check if PM2 is running

---

```
ssh ec2-user@51.21.219.35 pm2 status ```
```

### Solution:

- Ensure backend is running on port 3000
  - Check CORS configuration in backend allows frontend origin
- 

### Issue 2: "401 Unauthorized" Errors

**Cause:** Missing or invalid authentication token

**Check:** ```javascript console.log('Token:', localStorage.getItem('token'));  
console.log('Workspace ID:', localStorage.getItem('workspaceId')); ```

### Solution:

- Login again to get fresh token
  - Check token expiration (JWT)
  - Verify token is being sent in Authorization header
- 

### Issue 3: "Workspace Not Found" or "403 Forbidden"

**Cause:** Missing X-Workspace-ID header or invalid workspace access

**Check:** ````javascript console.log('Workspace ID:', localStorage.getItem('workspaceId')); console.log('Tenant ID:', localStorage.getItem('tenantId')); ````

### Solution:

- Ensure workspace\_id is stored after login
  - Verify user has access to requested workspace
  - Check multi-tenant isolation in backend
- 

### Issue 4: "Cannot read property of undefined"

**Cause:** API response structure different from expected

**Debug:** ````typescript try { const data = await workspaceApi.logistics.getDashboard(); console.log('API Response:', data);`

`// Check if response has expected structure if (!data || !data.success) { console.error('Unexpected response:', data); } } catch (error) { console.error('API Error:', error); } ````

### Solution:

- Log full API response to see actual structure
  - Update TypeScript interfaces to match backend response
  - Add defensive checks before accessing nested properties
- 



## Backend Response Structure (Standard)

---

All workspace endpoints return: ````json { "success": true, "data": { "stats": { ... }, "recentActivity": [ ... ], "alerts": [ ... ] }, "timestamp": "2025-11-16T..." } ````

Error responses: ``json { "success": false, "error": "Error message here", "code": "ERROR\_CODE", "timestamp": "2025-11-16T..." } ``

---

## ✅ Next Steps (Priority Order)

---

### Immediate (Today)

1. ✅ Test API Test Dashboard (`/api-test`)
2. ✅ Verify backend health endpoint works
3. ✅ Test login flow and token storage
4. ✅ Verify one module (Logistics) connects successfully

### Short Term (This Week)

5. ⌚ Connect Sales module to backend
6. ⌚ Connect Financial module to backend
7. ⌚ Connect HR module to backend
8. ⌚ Connect Inventory module to backend
9. ⌚ Test all CRUD operations (Create, Read, Update, Delete)

### Medium Term (Next Week)

10. ⌚ Replace all remaining mock data
  11. ⌚ Add loading states for all API calls
  12. ⌚ Add error boundaries for failed API calls
  13. ⌚ Implement data caching (React Query or SWR)
  14. ⌚ Add retry logic for failed requests
- 

## 🔍 Debugging Tips

---

### Enable API Logging

Add this to your component: ``typescript import { API\_BASE\_URL } from '.././services/api.service';

```
console.log('API Base URL:', API_BASE_URL); console.log('Token:',
localStorage.getItem('token')); console.log('Workspace ID:',
localStorage.getItem('workspaceId')); ``
```

### Monitor Network Requests

1. Open Chrome DevTools (F12)
2. Go to **Network** tab
3. Filter by **XHR** or **Fetch**
4. Click on request to see: - Request headers (check Authorization) - Response body (check data structure) - Status code (200, 401, 500, etc.)

### Test API Directly (Postman/Insomnia)

`` GET <http://51.21.219.35:3000/api/workspaces/logistics/dashboard>

Headers: Authorization: Bearer X-Workspace-ID: Content-Type: application/json ``



### Need Help?

If you encounter issues:

1. Check API Test Dashboard first (`/api-test`)
2. Review browser console for errors
3. Check Network tab in DevTools
4. Verify backend is running: `pm2 status`
5. Check backend logs: `pm2 logs`

---

**Last Updated:** November 16, 2025

**Status:** API Service Created, Testing Phase

**Next:** Connect all modules to real backend data

# DATABASE MIGRATION SUCCESS

---

## Database Migration SUCCESS - November 12, 2024

---

### MIGRATION COMPLETE

---

**Time:** 14:48 UTC

**Database:** aetheros-erp-db.cxoqqoowwgxt.eu-north-1.rds.amazonaws.com

**Status:** ALL TABLES CREATED 

---

### Created Objects

---

#### Schemas (3)

-  `sales`
-  `logistics`
-  `financial`

#### Tables (5)

-  `sales.customers` (1 row - 4PL.COM)
  -  `logistics.processed\_documents` (0 rows)
  -  `logistics.loads` (0 rows)
  -  `financial.invoices` (0 rows)
  -  `financial.invoice\_line\_items` (0 rows)
-

## Migration Output

```
🔄 Starting database migration...

📁 Creating schemas...
✅ Schemas created

📄 Creating sales.customers table...
✅ sales.customers created

📄 Creating logistics.processed_documents table...
✅ logistics.processed_documents created

📄 Creating logistics.loads table...
✅ logistics.loads created

📄 Creating financial.invoices table...
✅ financial.invoices created

📄 Creating financial.invoice_line_items table...
✅ financial.invoice_line_items created

👤 Inserting sample customer...
✅ Sample customer created with ID: 1

🔍 Verifying tables...

📊 Created tables:
• financial.invoice_line_items
• financial.invoices
• logistics.loads
• logistics.processed_documents
• sales.customers

👥 Total customers: 1

=====
✅ MIGRATION COMPLETE!
=====

🎉 Database is ready!
```

## NEXT STEP: Fix Backend Schema References

**Problem:** Backend queries `customers` but table is `sales.customers`

**Solution:** Update backend SQL queries to include schema prefix:

```
// Update in: backend/src/controllers/sales.controller.ts
FROM customers c // OLD ❌
FROM sales.customers c // NEW ✅
```

### Command to fix and restart:

```
ssh -i ~/.ssh/aetheros-aws.pem ec2-user@51.21.219.35
cd /home/ec2-user/backend/src/controllers
Edit sales.controller.ts
pm2 restart all
```

---

## Test Verification

Run this to verify everything:

```
ssh -i ~/.ssh/aetheros-aws.pem ec2-user@51.21.219.35 "node verify-db.js"
```

Expected output:

```
✅ Connected successfully!
Schemas found: 3
 - financial
 - logistics
 - sales
Tables found: 5
Customers found: 1
 - 4PL.COM - Logistical Solutions Provider (TAUFEEQ PETERSEN)
```

---

## What's Working

- ✅ RDS database accessible
- ✅ All schemas created
- ✅ All tables created with proper foreign keys
- ✅ Sample customer data inserted
- ✅ Backend server running (port 3000)
- ✅ Frontend deployed to S3

## What Needs Fixing

---

- ✗ Backend SQL queries need schema prefix ( `sales.customers` )
- ✗ OCR extraction needs testing with live data

**Next:** Update backend code to fix schema references, then test full workflow!